
FIP INF112

Compte rendu de l'itération 1

Réseau social d'opinions – BibTel

Mohammed-Amine Ziani & Wissam Lahouassa

Table des matières

1	Introduction	2
2	Conception UML	2
2.1	Diagramme de classes	2
2.2	Diagrammes de séquence	3
2.2.1	Scénario 1 – Ajout d’un membre (cas nominal)	3
3	Choix d’implémentation	4
3.1	Classe <code>Member</code>	4
3.2	Classe <code>SocialNetwork</code>	5
4	Dossier de tests	6
4.1	Scénarios de test – <code>InitTest</code>	6
4.2	Scénarios de test – <code>AddMemberTest</code>	7
4.2.1	Groupe 1 – Paramètres invalides (<code>BadEntryException</code> attendue) . .	7
4.2.2	Groupe 2a – Ajout nominal (<code>addMemberOKTest</code>)	7
4.2.3	Groupe 2b – Doubleton (<code>MemberAlreadyExistsException</code> attendue)	7
4.2.4	Groupe 3 – Intégrité du réseau	8
4.3	Résultats d’exécution	8
4.3.1	Analyse détaillée des résultats	8
5	Bilan de l’itération	9
6	Conclusion	9

1 Introduction

Ce document constitue le compte rendu de la première itération du projet *SocialNetwork*, développé dans le cadre de l'unité d'enseignement **FIP INF112 – GL & Gestion de projet informatique** à IMT Atlantique (année scolaire 2024-2025).

L'objectif de cette première itération est d'implémenter les fonctionnalités de gestion des **membres** du réseau social BibTel, conformément aux spécifications de l'API imposée par le client (interface `ISocialNetwork`). Plus précisément, les méthodes visées sont :

- `addMember()` – ajout d'un nouveau membre,
- `nbMembers()` – retourne le nombre de membres inscrits,
- `toString()` – affichage de l'état courant du réseau.

La démarche adoptée suit le principe du **développement dirigé par les tests** : les scénarios de test ont été préparés *avant* l'implémentation, et le code a ensuite été écrit pour les satisfaire.

2 Conception UML

2.1 Diagramme de classes

La figure 1 présente le diagramme de classes conçu pour cette itération. L'interface `ISocialNetwork` est implémentée par `SocialNetwork`, qui agrège une liste de `Member`. Les exceptions `BadEntryException` et `MemberAlreadyExistsException` sont lancées par `SocialNetwork`.

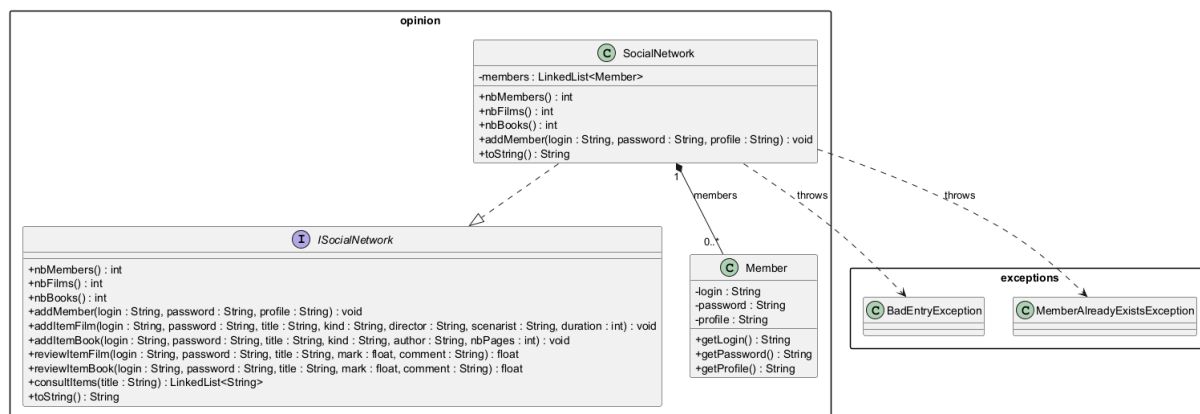


FIGURE 1 – Diagramme de classes – Itération 1

Points de conception notables :

- La classe `Member` est déclarée avec une visibilité *package-private* afin d'encapsuler les détails internes dans le package `opinion` et d'empêcher tout accès extérieur non contrôlé.
- La navigabilité de l'association `SocialNetwork` → `Member` est unidirectionnelle : seul `SocialNetwork` connaît ses membres, pas l'inverse. Cela évite le couplage inutile.

- Les attributs de `Member` sont tous `private` ; les accesseurs (`getLogin()`, `getPassword()`, `getProfile()`) ont une visibilité *package-private* pour la même raison d'encapsulation.

2.2 Diagrammes de séquence

Deux scénarios détaillés ont été modélisés.

2.2.1 Scénario 1 – Ajout d'un membre (cas nominal)

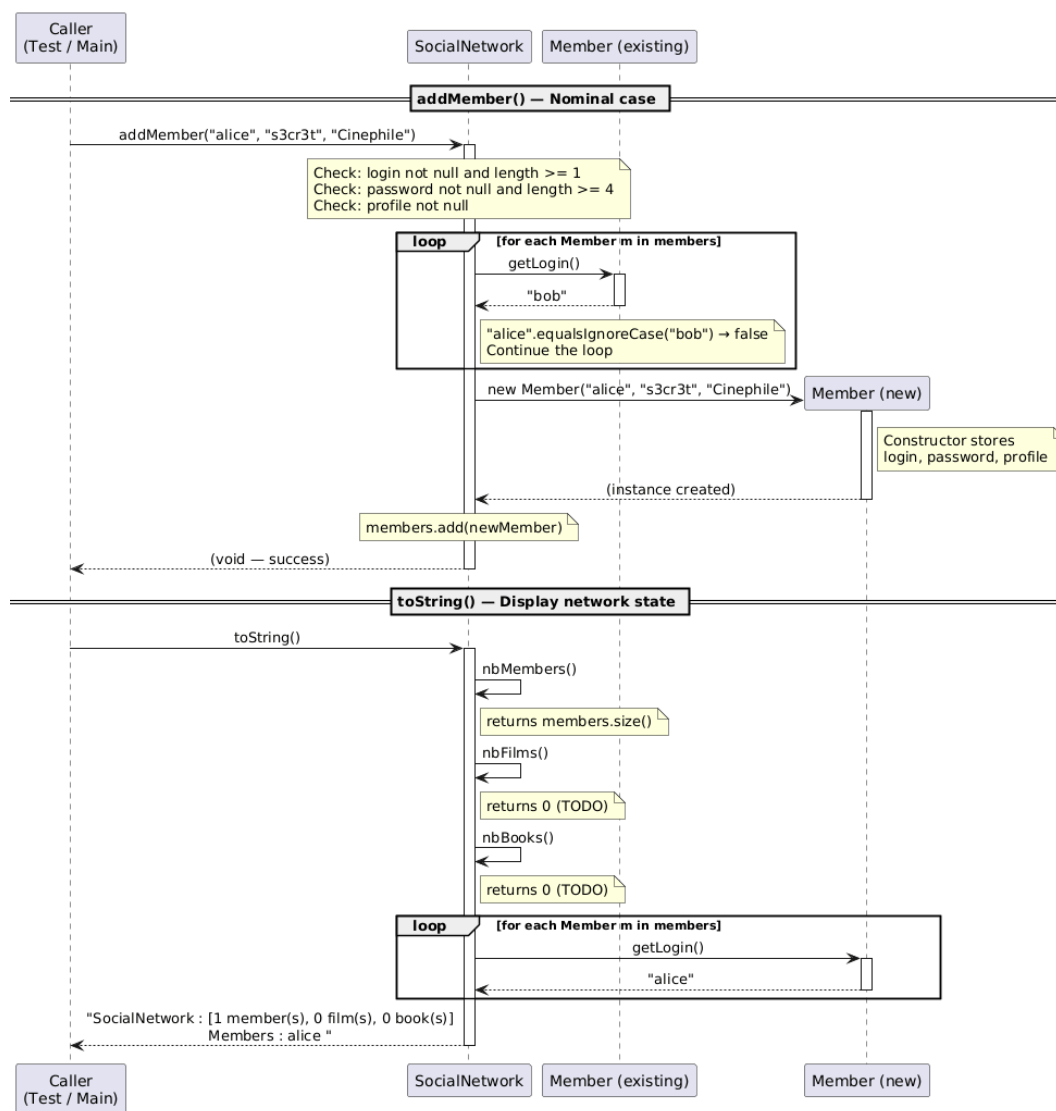


FIGURE 2 – Diagramme de séquence – `addMember()` cas nominal et `toString()`

Le diagramme illustre :

1. La validation des paramètres d'entrée par `SocialNetwork`.
2. La boucle de détection de doublon (comparaison insensible à la casse et aux espaces).
3. La création de l'instance `Member` et son insertion dans la liste interne.
4. L'appel à `toString()` qui délègue aux méthodes d'inspection et itère sur les membres.

3 Choix d'implémentation

3.1 Classe Member

```
1 package opinion;
2
3 /**
4  * Represents a member of the SocialNetwork.
5  */
6 class Member {
7
8     private String login;
9     private String password;
10    private String profile;
11
12    /**
13     * @param login    the member's login
14     * @param password the member's password
15     * @param profile  the member's profile description
16     */
17    Member(String login, String password, String profile) {
18        this.login    = login;
19        this.password = password;
20        this.profile  = profile;
21    }
22
23    String getLogin()    { return login;    }
24    String getPassword() { return password; }
25    String getProfile()  { return profile; }
26 }
```

Listing 1 – Classe Member.java – package opinion

Justifications des choix :

Visibilité *package* La classe est déclarée sans modificateur (`class Member`), ce qui la rend accessible uniquement dans le package `opinion`. Cela respecte le principe d'encapsulation : les classes extérieures (notamment les tests et l'IHM) ne manipulent les membres qu'au travers de l'interface `ISocialNetwork`.

Attributs *private* Les trois champs (`login`, `password`, `profile`) sont privés. Les accesseurs correspondants ont une visibilité *package-private* : ils sont utilisables par `SocialNetwork` mais invisibles depuis l'extérieur du package.

Pas d'héritage `Member` n'étend aucune classe et n'implémente aucune interface : l'approche minimaliste recommandée par les bonnes pratiques du projet est respectée.

3.2 Classe SocialNetwork

```

1 package opinion;
2
3 import java.util.LinkedList;
4 import exceptions.BadEntryException;
5 import exceptions.MemberAlreadyExistsException;
6
7 public class SocialNetwork implements ISocialNetwork {
8
9     private LinkedList<Member> members = new LinkedList<Member>();
10
11     @Override
12     public int nbMembers() {
13         return members.size();
14     }
15
16     @Override
17     public void addMember(String login, String password, String
        profile)
18         throws BadEntryException,
19             MemberAlreadyExistsException {
20
21         // --- Validation des parametres ---
22         if (login == null || login.trim().length() < 1)
23             throw new BadEntryException("login invalide");
24         if (password == null || password.trim().length() < 4)
25             throw new BadEntryException("password invalide");
26         if (profile == null)
27             throw new BadEntryException("profile invalide");
28
29         // --- Detection de doublon ---
30         for (Member m : members) {
31             if (login.trim().equalsIgnoreCase(m.getLogin().trim()))
32                 throw new MemberAlreadyExistsException();
33         }
34
35         // --- Ajout du nouveau membre ---
36         members.add(new Member(login, password, profile));
37
38     @Override
39     public String toString() {
40         String s = "SocialNetwork : [" + nbMembers() + " membre
            (s), "
41             + nbFilms() + " film(s), " + nbBooks() + "
            livre(s)]\n";
42         s += "Membres : ";
43         for (Member m : members) {
44             s += m.getLogin() + " ";

```

```

45         }
46         return s;
47     }
48
49 }
```

Listing 2 – Classe `SocialNetwork.java` – méthodes implémentées**Justifications des choix :****Validation stricte**

- Le login est rejeté s'il est `null` ou si sa version sans espaces de bord (`trim()`) a une longueur inférieure à 1.
- Le mot de passe est rejeté s'il est `null` ou si `trim()` donne moins de 4 caractères. Cela filtre les mots de passe du type " qwd " (3 caractères utiles).
- Le profil est rejeté uniquement s'il est `null` ; une chaîne vide est acceptée (la spécification ne l'interdit pas).

Comparaison

L'expression `login.trim().equalsIgnoreCase(m.getLogin().trim())` garantit qu'un login comme "anToine" ou " Antoine " est bien détecté comme doublon de "Antoine".

Ordre

Les vérifications `BadEntryException` sont effectuées *avant* la boucle de détection de doublon. Ainsi, si un paramètre est invalide *et* qu'un doublon existe, l'exception levée est `BadEntryException`, ce qui est cohérent avec la spécification.

4 Dossier de tests

4.1 Scénarios de test – `InitTest`

TABLE 1 – Scénarios `InitTest` – Construction d'un réseau vide

ID	Description	Résultat attendu
0.1	<code>nbMembers()</code> sur un réseau nouvellement créé	0
0.2	<code>nbBooks()</code> sur un réseau nouvellement créé	0
0.3	<code>nbFilms()</code> sur un réseau nouvellement créé	0

4.2 Scénarios de test – AddMemberTest

4.2.1 Groupe 1 – Paramètres invalides (BadEntryException attendue)

TABLE 2 – Tests de rejet de paramètres invalides

ID	Appel	Exception attendue
1.1	<code>addMember(null, "qsdfgh", "")</code>	<code>BadEntryException</code> (login null)
1.2	<code>addMember(" ", "qsdfgh", "")</code>	<code>BadEntryException</code> (login sans caractère non-espace)
1.3	<code>addMember("B", null, "")</code>	<code>BadEntryException</code> (password null)
1.4	<code>addMember("B", " qwd ", "")</code>	<code>BadEntryException</code> (password < 4 car. utiles)
1.5	<code>addMember("BBBB", "bbbb", null)</code>	<code>BadEntryException</code> (profile null)

Note : les tildes symbolisent des espaces dans le tableau.

Pour chaque test de ce groupe, la méthode auxiliaire `addMemberBadEntryTest` vérifie également que `nbMembers()` n'a **pas** changé après le rejet : l'état du réseau doit rester intact.

4.2.2 Groupe 2a – Ajout nominal (addMemberOKTest)

TABLE 3 – Tests d'ajout nominal de membres

ID	Appel	Vérification
2.1a	<code>addMember("Paul", "paul", "lecteur impulsif")</code>	<code>nbMembers()</code> passe de 0 à 1
2.1b	<code>addMember("Antoine", "antoine", "grand amoureux...")</code>	<code>nbMembers()</code> passe de 1 à 2
2.1c	<code>addMember("Alice", "alice", "passionnée de BD")</code>	<code>nbMembers()</code> passe de 2 à 3

4.2.3 Groupe 2b – Doubleton (MemberAlreadyExistsException attendue)

TABLE 4 – Tests de détection de doublon

ID	Appel	Cas testé
2.2	<code>addMember("Paul", "abcdefghij", "")</code>	Login du premier membre (doublon exact)
2.3	<code>addMember("Alice", "abcdefghij", "")</code>	Login du dernier membre (doublon exact)
2.4	<code>addMember("anToine", "abcdefghij", "")</code>	Même login, casse différente
2.5	<code>addMember(" Antoine ", "abcdefghij", "")</code>	Login encadré d'espaces
2.6	<code>addMember("An"+"toi"+"ne", "abcdefghij", "")</code>	Login obtenu par concaténation de littéraux

4.2.4 Groupe 3 – Intégrité du réseau

TABLE 5 – Vérification de la non-modification des compteurs films/livres

ID	Description	Valeur attendue
3.1	nbFilms() après tous les appels à addMember()	0
3.2	nbBooks() après tous les appels à addMember()	0

4.3 Résultats d'exécution

Résultat global – Suite de tests SocialNetworkTest

```
Testing brand new Social Networks...
Here is the newly created SocialNetwork :
SocialNetwork : [0 membre(s), 0 film(s), 0 livre(s)]
Membres :
InitTest : 3 tests, 0 erreur(s)
```

```
Testing addMember()
Final state of the social network :
SocialNetwork : [3 membre(s), 0 film(s), 0 livre(s)]
Membres : Paul Antoine Alice
AddMemberTest : 15 tests, 0 erreur(s)
```

```
Global tests results :
18 tests, 0 erreur(s)
```

4.3.1 Analyse détaillée des résultats

TABLE 6 – Récapitulatif des résultats par groupe de tests

Classe	Groupe	Tests	OK	KO
InitTest	Construction réseau vide	3	3	0
	BadEntry (param. invalides)	5	5	0
AddMemberTest	Ajout nominal	3	3	0
	MemberAlreadyExists (doublon)	5	5	0
	Intégrité films/livres	2	2	0
Total		18	18	0

5 Bilan de l'itération

Objectifs atteints – 18/18 tests passés

- ✓ `nbMembers()` retourne correctement la taille de la liste interne.
- ✓ `addMember()` valide tous les paramètres d'entrée.
- ✓ `addMember()` détecte les doublons (insensible à la casse et aux espaces).
- ✓ `addMember()` crée et stocke correctement les nouveaux membres.
- ✓ `toString()` affiche l'état complet du réseau (membres, films, livres).
- ✓ Aucun effet de bord sur `nbFilms()` et `nbBooks()`.

6 Conclusion

Cette première itération a permis d'implémenter et de valider la gestion des membres dans le réseau social **SocialNetwork**. L'ensemble des 18 tests de la suite automatisée passent sans erreur, ce qui confirme la conformité du code produit aux spécifications de l'API cliente.

La démarche de développement dirigé par les tests s'est avérée efficace : les scénarios de test, préparés avant l'implémentation, ont guidé les choix de conception (notamment la validation des paramètres et la comparaison insensible à la casse) et ont immédiatement mis en évidence les erreurs lors du développement.

Les fonctionnalités relatives aux films, aux livres et aux avis seront développées dans les itérations suivantes, en s'appuyant sur les bases consolidées dans cette itération.